

Fallback Pattern

1. Overview

The **Fallback Pattern** is a fault-tolerance strategy where, when the primary service/path fails, the system detects the issue and executes an **alternative code path** so the user still receives a usable response.

Instead of returning:

None

500 Internal Server Error

Return:

- Cached data
- Default response
- Reduced functionality
- Alternate provider
- Static content
- Graceful message

2. Core Idea

None

Primary path fails

↓

Switch to backup behavior

↓

System remains usable

Goal:

- Improve availability
- Better user experience
- Reduce total outages

3. Why Fallback Is Needed

Distributed systems depend on many components:

- Databases
- Search services
- Third-party APIs
- Recommendation engines
- Maps APIs
- Payment gateways

If one dependency fails, entire request shouldn't always fail.

Fallback allows graceful degradation.

4. Example: Nearby Bagel Cafes

Primary flow:

None

Mobile App → Backend Service → Nearby Cafes API

If backend fails:

Fallback options:

Option A: Cached Results

None

Show last known nearby cafes

Option B: Third-Party Maps API

Use:

- Google Maps
- Bing Maps

May be less customized, but usable.

5. Common Types of Fallback

A. Cached Response Fallback

If live service fails:

None

Return cached data

Used for:

- Product catalog
- Search suggestions
- Nearby places
- User profile

B. Default Response Fallback

None

Recommendations unavailable

→ Show popular products

C. Alternate Provider Fallback

Primary SMS provider fails:

None

Switch Twilio → Another provider

D. Static Content Fallback

If CMS down:

None

Serve pre-generated homepage

E. Queue for Later Processing

If email service fails:

None

Store request and send later

6. Example: E-Commerce Recommendations

Primary:

None

Recommendation Engine

Failure:

Fallback:

None

Show Best Sellers

Show Trending Products

User still sees content.

7. Example: Payment System

Primary payment gateway down.

Fallback options:

None

Gateway A fails



Use Gateway B

Or:

None

Accept COD / Pay later

8. Example: Search Service

Search index unavailable.

Fallback:

None

Return recent popular searches

Return category browsing only

9. Fallback Flow Diagram

None

Request



Try Primary Service



Success? ---- Yes → Return Normal Response



No



Trigger Fallback



Return Alternate Response

10. Important Warning: Fallback Can Also Fail

Fallback systems are still systems.

Examples:

- Cache miss
- Backup provider down
- Stale data too old
- Third-party quota exceeded
- Backup slower than primary

Need to design for fallback reliability too.

11. Why Amazon Often Avoids Fallback Overuse

Fallbacks can create hidden complexity:

- Rarely tested paths
- Unexpected inconsistent behavior
- Increased latency
- Cascading calls to backup systems
- Operational confusion

Instead many large systems prefer:

- Simpler graceful degradation
- Fast failure
- Timeouts
- Retries
- Capacity planning
- Circuit breakers

Important interview insight: fallback is useful, but not always ideal.

12. Fallback vs Retry

Retry

Try same service again.

None

Temporary timeout → retry

Fallback

Use alternate path.

None

Service still unavailable → use cache

Often used together.

13. Fallback vs Circuit Breaker

Circuit Breaker

Stops repeated calls to failing service.

Fallback

Provides alternate response after breaker opens.

Example:

None

Breaker open

↓

Use fallback cache

14. Real-World Examples

Netflix

If recommendations fail:

→ show continue watching / trending

Uber

If surge calculator fails:

→ show approximate fare

Maps App

If live traffic unavailable:

→ show route without traffic data

Banking

If statement service down:

→ show yesterday snapshot

15. Design Considerations

A. Business Criticality

Is stale data acceptable?

Use Case	Fallback Safe?
----------	----------------

Product catalog	Yes
News feed	Yes
Bank balance	Risky
OTP verification	No
Inventory count	Limited

B. Freshness

Cached data may be outdated.

Need TTL.

C. User Transparency

Sometimes show:

None

Showing last updated results

D. Cost

Backup providers may cost more.

E. Simplicity

Too many fallback layers create chaos.

16. Interview Answer

Q: How would you handle dependency outage in HLD?

Answer:

I'd define fallback behavior based on business criticality. For non-critical features, I may serve cached data or defaults. For provider integrations, I may switch to a secondary provider. I'd combine this with timeouts, retries, and circuit breakers, while keeping fallback logic simple and tested.

17. Best Practices

- Keep fallback simple
- Test fallback regularly
- Use for noncritical paths first
- Add timeouts
- Combine with circuit breaker
- Track fallback activation metrics
- Inform users if stale data matters
- Avoid infinite fallback chains

18. When NOT to Use Fallback

Avoid for high-risk operations:

- Money transfer amount
- Real-time stock trades
- Security authentication
- Exact inventory reservations
- Medical critical data

Better to fail clearly than give wrong data.

19. Short Revision Notes

None

Fallback = Alternative path when primary fails

Examples:

- Cached data
- Default response
- Secondary provider
- Static page

Use with:

Retry + Timeout + Circuit Breaker

Risk:

Fallback can fail too